

DATA ANALYST INTERNSHIP

Task 4: SQL for Data Analysis

Submitted by: Sandeep Verma

Tool Used: MySQL Community Server 8.0 + MySQL Workbench

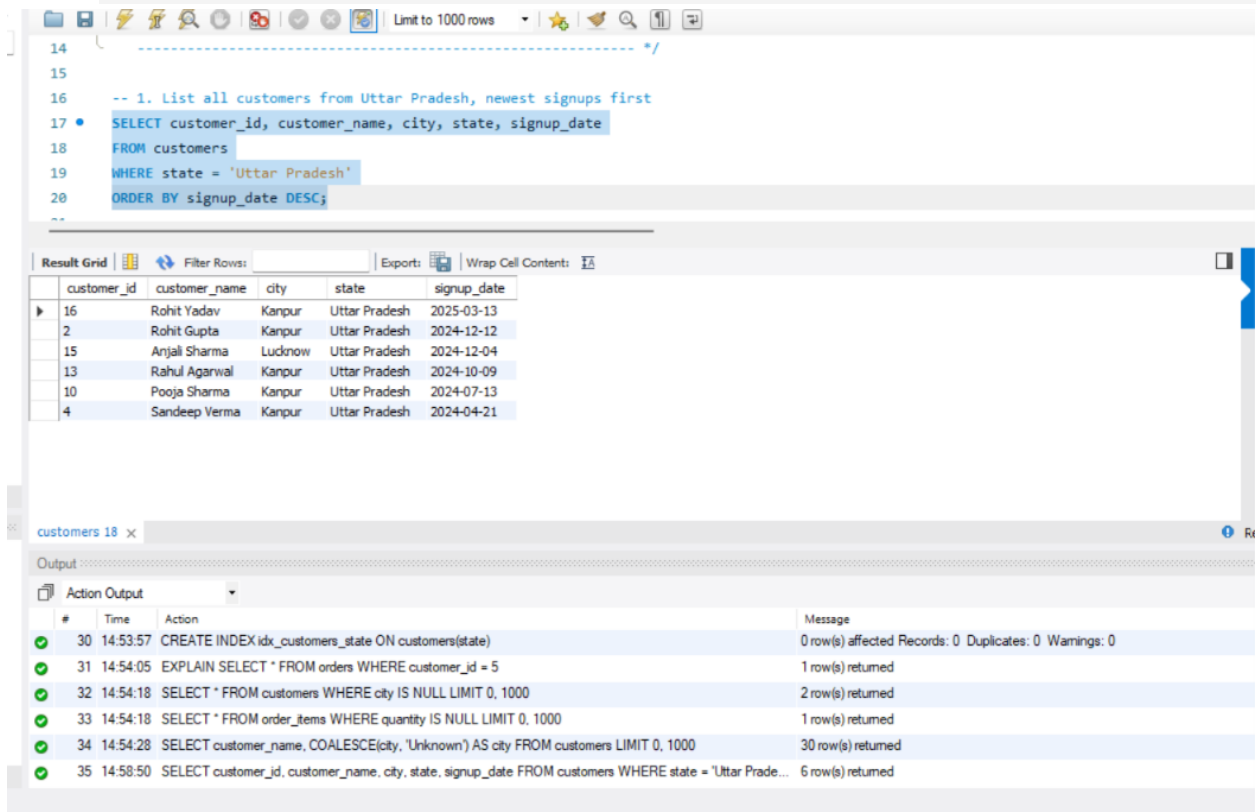
Database: ecommerce_db (Customers, Products, Orders, Order Items)

Objective	Use SQL queries to extract and analyze data from a database.
Tools	MySQL Community Server, MySQL Workbench
Deliverables	SQL queries in a SQL file + screenshots of output
Dataset	Custom-built Ecommerce database (customers, products, orders, order_items)
Outcome	Learn to manipulate and query structured data using SQL

a. SELECT, WHERE, ORDER BY, GROUP BY

Query 1: List all customers from Uttar Pradesh, newest signups first

```
SELECT customer_id, customer_name, city, state, signup_date
FROM customers
WHERE state = 'Uttar Pradesh'
ORDER BY signup_date DESC;
```



The screenshot displays the MySQL Workbench interface. The query editor at the top shows the SQL query for Query 1. Below the editor, the 'Result Grid' tab is active, showing the results of the query. The results are as follows:

customer_id	customer_name	city	state	signup_date
16	Rohit Yadav	Kanpur	Uttar Pradesh	2025-03-13
2	Rohit Gupta	Kanpur	Uttar Pradesh	2024-12-12
15	Anjali Sharma	Lucknow	Uttar Pradesh	2024-12-04
13	Rahul Agarwal	Kanpur	Uttar Pradesh	2024-10-09
10	Pooja Sharma	Kanpur	Uttar Pradesh	2024-07-13
4	Sandeep Verma	Kanpur	Uttar Pradesh	2024-04-21

Below the result grid, the 'Output' pane shows the 'Action Output' log, which includes the execution of the query and the number of rows returned (6 rows).

Screenshot: Output of Query 1 in MySQL Workbench

Query 2: List all products priced above 1000, cheapest first

```
SELECT product_id, product_name, category, price
FROM products
WHERE price > 1000
ORDER BY price ASC;
```

```

20 ORDER BY signup_date DESC;
21
22 -- 2. List all products priced above 1000, cheapest first
23 • SELECT product_id, product_name, category, price
24 FROM products
25 WHERE price > 1000
26 ORDER BY price ASC;

```

	product_id	product_name	category	price
▶	2	Bluetooth Headphones	Electronics	1299.00
	8	Backpack	Accessories	1599.00
	11	Running Shoes	Fashion	1999.00
	10	Smartwatch	Electronics	2499.00
	5	Study Table	Furniture	3499.00
	4	Office Chair	Furniture	4999.00

Screenshot: Output of Query 2 in MySQL Workbench

Query 3: Count how many orders exist per status

```

SELECT status, COUNT(*) AS total_orders
FROM orders
GROUP BY status
ORDER BY total_orders DESC;

```

Query 3: Count how many orders exist per status

```

26 ORDER BY price ASC;
27
28 -- 3. Count how many orders exist per status
29 • SELECT status, COUNT(*) AS total_orders
30 FROM orders
31 GROUP BY status
32 ORDER BY total_orders DESC;

```

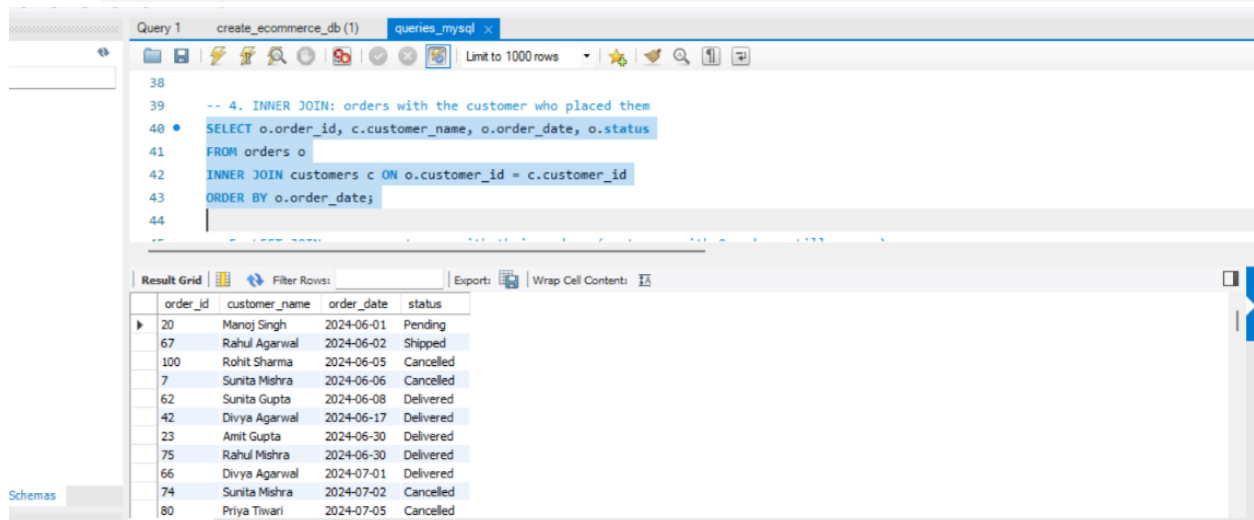
status	total_orders
Delivered	48
Cancelled	21
Shipped	17
Pending	14

Screenshot: Output of Query 3 in MySQL Workbench

b. JOINS (INNER, LEFT, RIGHT)

Query 4: INNER JOIN — orders with the customer who placed them

```
SELECT o.order_id, c.customer_name, o.order_date, o.status
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_date;
```



Query 1 create_ecommerce_db (1) queries_mysql

Limit to 1000 rows

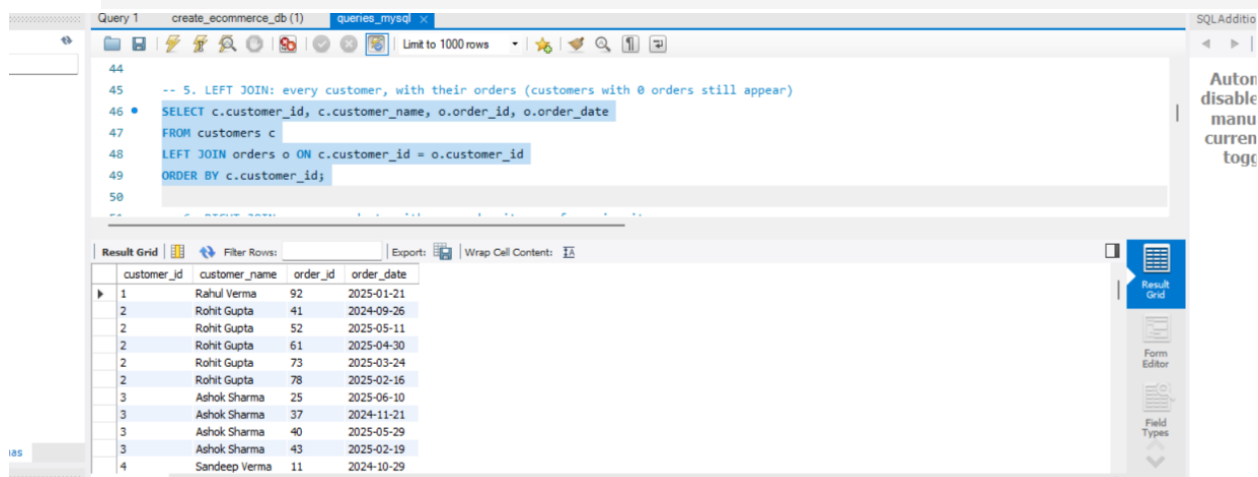
Result Grid

order_id	customer_name	order_date	status
20	Manoj Singh	2024-06-01	Pending
67	Rahul Agarwal	2024-06-02	Shipped
100	Rohit Sharma	2024-06-05	Cancelled
7	Sunita Mishra	2024-06-06	Cancelled
62	Sunita Gupta	2024-06-08	Delivered
42	Divya Agarwal	2024-06-17	Delivered
23	Amit Gupta	2024-06-30	Delivered
75	Rahul Mishra	2024-06-30	Delivered
66	Divya Agarwal	2024-07-01	Delivered
74	Sunita Mishra	2024-07-02	Cancelled
80	Priya Tiwari	2024-07-05	Cancelled

Screenshot: Output of Query 4 in MySQL Workbench

Query 5: LEFT JOIN — every customer, with their orders (customers with 0 orders still appear)

```
SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
ORDER BY c.customer_id;
```



Query 1 create_ecommerce_db (1) queries_mysql

Limit to 1000 rows

Result Grid

customer_id	customer_name	order_id	order_date
1	Rahul Verma	92	2025-01-21
2	Rohit Gupta	41	2024-09-26
2	Rohit Gupta	52	2025-05-11
2	Rohit Gupta	61	2025-04-30
2	Rohit Gupta	73	2025-03-24
2	Rohit Gupta	78	2025-02-16
3	Ashok Sharma	25	2025-06-10
3	Ashok Sharma	37	2024-11-21
3	Ashok Sharma	40	2025-05-29
3	Ashok Sharma	43	2025-02-19
4	Sandeep Verma	11	2024-10-29

Screenshot: Output of Query 5 in MySQL Workbench

Query 6: RIGHT JOIN — every product, with any order_items referencing it

```
SELECT p.product_name, oi.order_item_id, oi.quantity
FROM order_items oi
RIGHT JOIN products p ON oi.product_id = p.product_id
ORDER BY p.product_name;
```

50

51 - Execute the selected portion of the script or everything, if there is no selection, clicking it

52 • SELECT p.product_name, oi.order_item_id, oi.quantity

53 FROM order_items oi

54 RIGHT JOIN products p ON oi.product_id = p.product_id

55 ORDER BY p.product_name;

56

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	product_name	order_item_id	quantity
▶	Backpack	17	3
	Backpack	40	2
	Backpack	75	3
	Backpack	88	2
	Backpack	98	1
	Backpack	105	2
	Backpack	112	1
	Backpack	124	2
	Backpack	128	2
	Backpack	141	2
	Backpack	143	2

Result 23 x

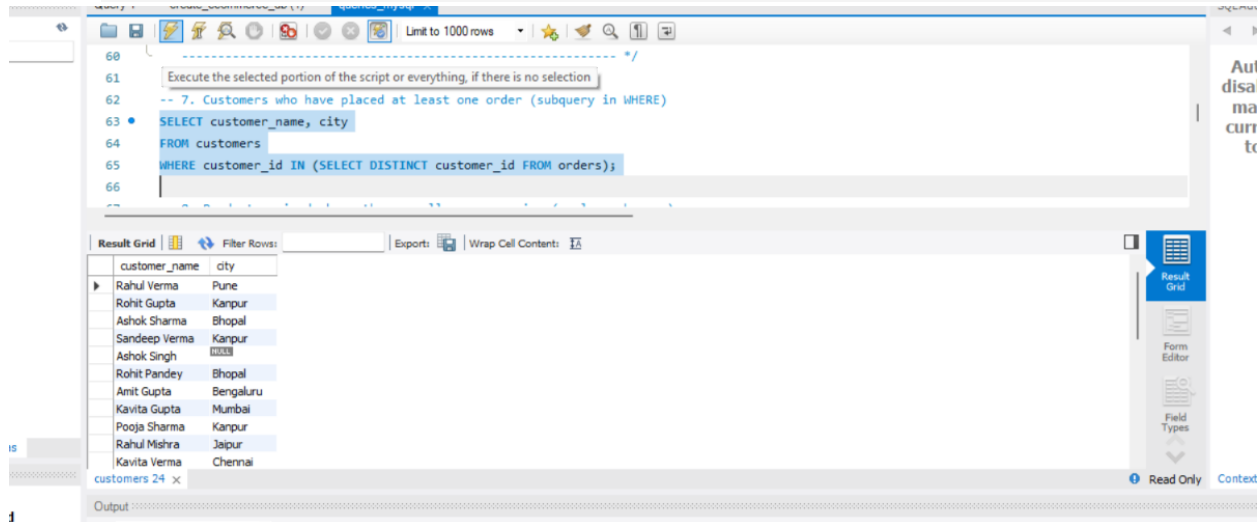
Output

Screenshot: Output of Query 6 in MySQL Workbench

c. Subqueries

Query 7: Customers who have placed at least one order (subquery in WHERE)

```
SELECT customer_name, city
FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
```



Screenshot: Output of Query 7 in MySQL Workbench

Query 8: Products priced above the overall average price (scalar subquery)

```
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

Execute the selected portion of the script or everything, if there is no selection

```

66
67 -- 8. Products priced above the overall average price (scalar subquery)
68 • SELECT product_name, price
69 FROM products
70 WHERE price > (SELECT AVG(price) FROM products)
71 ORDER BY price DESC;
72

```

Result Grid

	product_name	price
▶	Office Chair	4999.00
	Study Table	3499.00
	Smartwatch	2499.00
	Running Shoes	1999.00
	Backpack	1599.00

products 25 x

Output

Screenshot: Output of Query 8 in MySQL Workbench

Query 9: Top 5 customers by total amount spent (subquery in FROM)

```

SELECT customer_name, total_spent
FROM (
  SELECT c.customer_name, SUM(oi.quantity * oi.unit_price) AS total_spent
  FROM customers c
  JOIN orders o ON c.customer_id = o.customer_id
  JOIN order_items oi ON o.order_id = oi.order_id
  GROUP BY c.customer_id, c.customer_name
) AS customer_totals
ORDER BY total_spent DESC
LIMIT 5;

```

Query 1 create_ecommerce_db (1) queries_mysql x

Limit to 1000 rows

```

78 FROM customers c
79 Execute the selected portion of the script or everything, if there is no selection
80 JOIN order_items oi ON o.order_id = oi.order_id
81 GROUP BY c.customer_id, c.customer_name
82 ) AS customer_totals
83 ORDER BY total_spent DESC
84 LIMIT 5;

```

Result Grid

	customer_name	total_spent
▶	Ashok Sharma	58673.00
	Rohit Gupta	47774.00
	Rahul Mishra	43177.00
	Rohit Kumar	42672.00
	Divya Agarwal	42466.00

Result 26 x

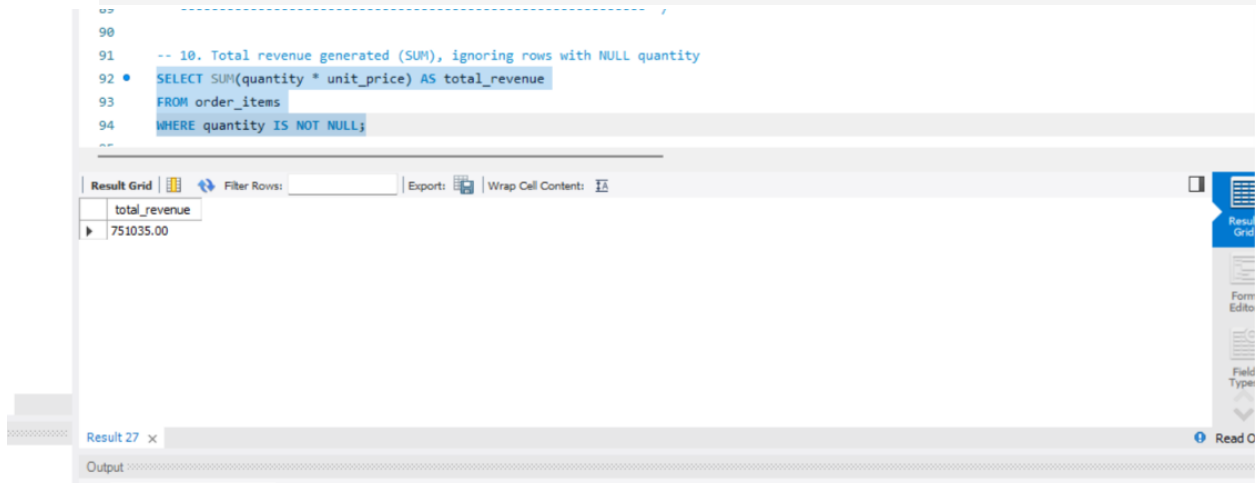
Read

Screenshot: Output of Query 9 in MySQL Workbench

d. Aggregate Functions (SUM, AVG)

Query 10: Total revenue generated (SUM), ignoring rows with NULL quantity

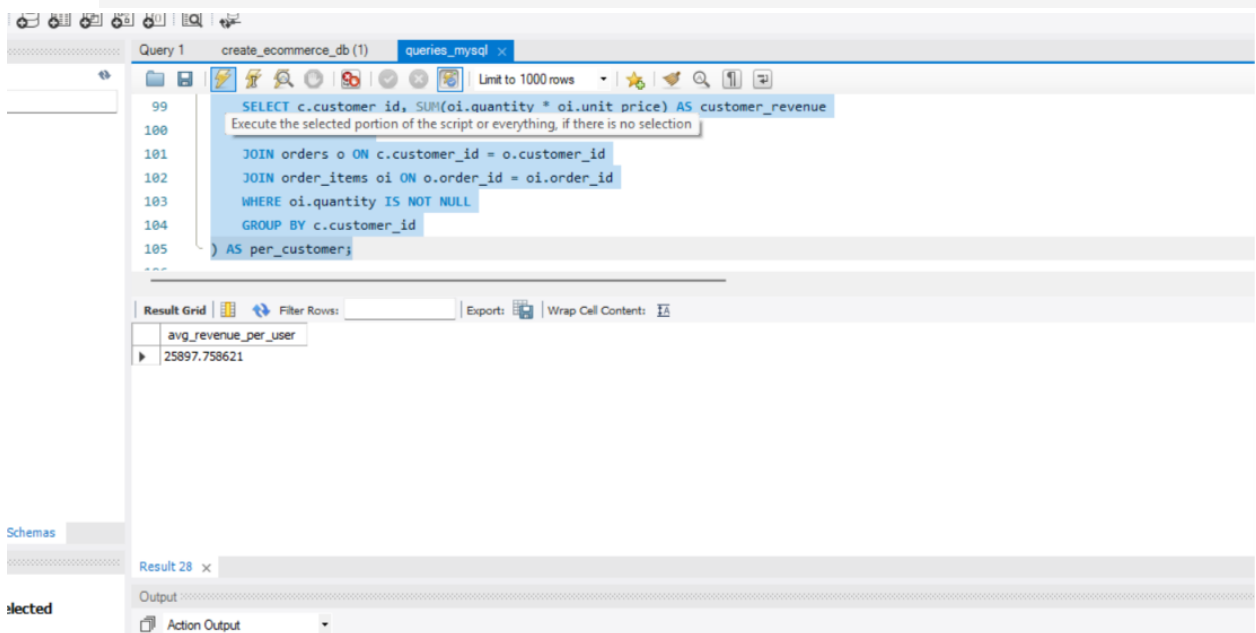
```
SELECT SUM(quantity * unit_price) AS total_revenue
FROM order_items
WHERE quantity IS NOT NULL;
```



Screenshot: Output of Query 10 in MySQL Workbench

Query 11: Average revenue per user (SUM per customer, then AVG across customers)

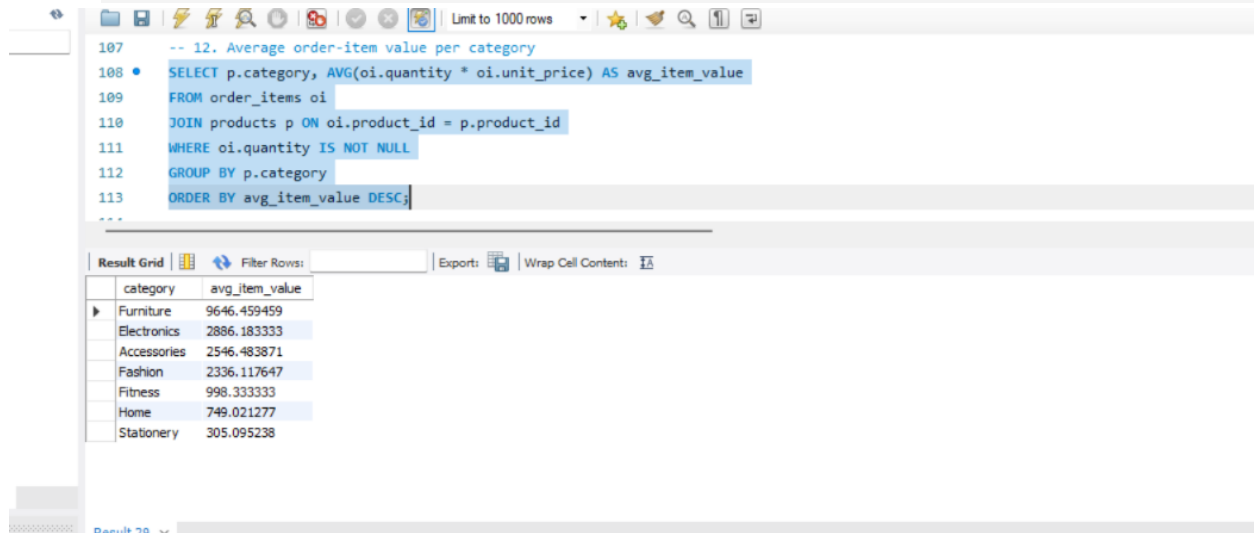
```
SELECT AVG(customer_revenue) AS avg_revenue_per_user
FROM (
  SELECT c.customer_id, SUM(oi.quantity * oi.unit_price) AS customer_revenue
  FROM customers c
  JOIN orders o ON c.customer_id = o.customer_id
  JOIN order_items oi ON o.order_id = oi.order_id
  WHERE oi.quantity IS NOT NULL
  GROUP BY c.customer_id
) AS per_customer;
```



Screenshot: Output of Query 11 in MySQL Workbench

Query 12: Average order-item value per category

```
SELECT p.category, AVG(oi.quantity * oi.unit_price) AS avg_item_value
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
WHERE oi.quantity IS NOT NULL
GROUP BY p.category
ORDER BY avg_item_value DESC;
```



The screenshot shows the MySQL Workbench interface. The query editor at the top displays the following SQL query:

```
-- 12. Average order-item value per category
SELECT p.category, AVG(oi.quantity * oi.unit_price) AS avg_item_value
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
WHERE oi.quantity IS NOT NULL
GROUP BY p.category
ORDER BY avg_item_value DESC;
```

Below the query editor, the Result Grid shows the output of the query. The results are as follows:

category	avg_item_value
Furniture	9646.459459
Electronics	2886.183333
Accessories	2546.483871
Fashion	2336.117647
Fitness	998.333333
Home	749.021277
Stationery	305.095238

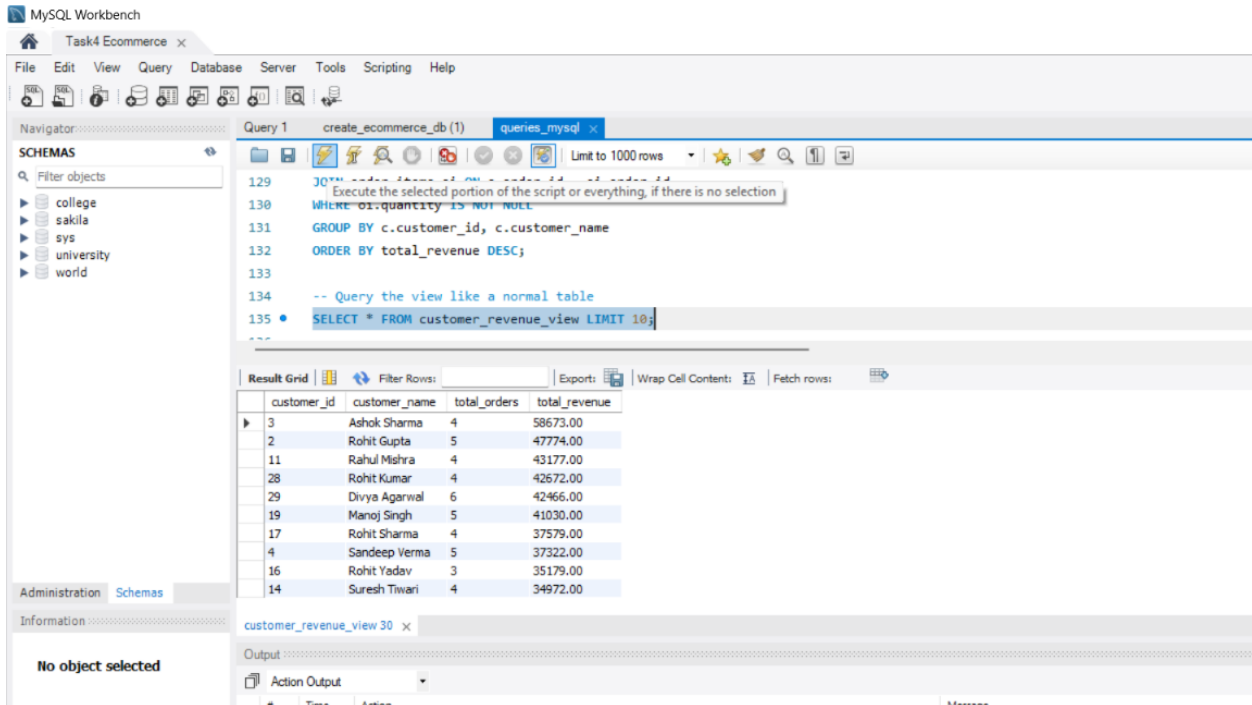
Screenshot: Output of Query 12 in MySQL Workbench

e. Create Views for Analysis

View: customer_revenue_view

```
CREATE VIEW customer_revenue_view AS
SELECT c.customer_id,
       c.customer_name,
       COUNT(DISTINCT o.order_id) AS total_orders,
       SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE oi.quantity IS NOT NULL
GROUP BY c.customer_id, c.customer_name
ORDER BY total_revenue DESC;

-- Query the view like a normal table
SELECT * FROM customer_revenue_view LIMIT 10;
```



The screenshot shows the MySQL Workbench interface. The 'Query 1' window displays the SQL code for creating the view and querying it. The 'Result Grid' shows the output of the query, listing the top 10 customers by total revenue.

customer_id	customer_name	total_orders	total_revenue
3	Ashok Sharma	4	58673.00
2	Rohit Gupta	5	47774.00
11	Rahul Mishra	4	43177.00
28	Rohit Kumar	4	42672.00
29	Divya Agarwal	6	42466.00
19	Manoj Singh	5	41030.00
17	Rohit Sharma	4	37579.00
4	Sandeep Verma	5	37322.00
16	Rohit Yadav	3	35179.00
14	Suresh Tiwari	4	34972.00

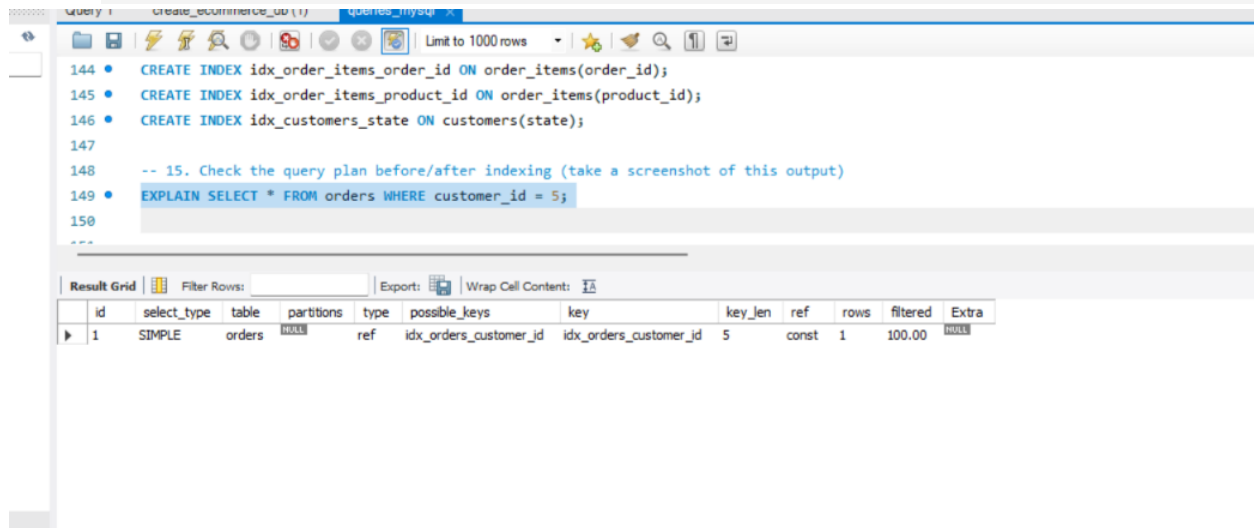
Screenshot: customer_revenue_view created and queried successfully

f. Optimize Queries with Indexes

Creating indexes + checking the query plan

```
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_order_items_order_id ON order_items(order_id);
CREATE INDEX idx_order_items_product_id ON order_items(product_id);
CREATE INDEX idx_customers_state ON customers(state);
```

```
-- Check the query plan after indexing
EXPLAIN SELECT * FROM orders WHERE customer_id = 5;
```



The screenshot shows a MySQL query editor window with the following SQL commands:

```
144 • CREATE INDEX idx_order_items_order_id ON order_items(order_id);
145 • CREATE INDEX idx_order_items_product_id ON order_items(product_id);
146 • CREATE INDEX idx_customers_state ON customers(state);
147
148 -- 15. Check the query plan before/after indexing (take a screenshot of this output)
149 • EXPLAIN SELECT * FROM orders WHERE customer_id = 5;
150
```

Below the SQL editor, the 'Result Grid' shows the output of the EXPLAIN query:

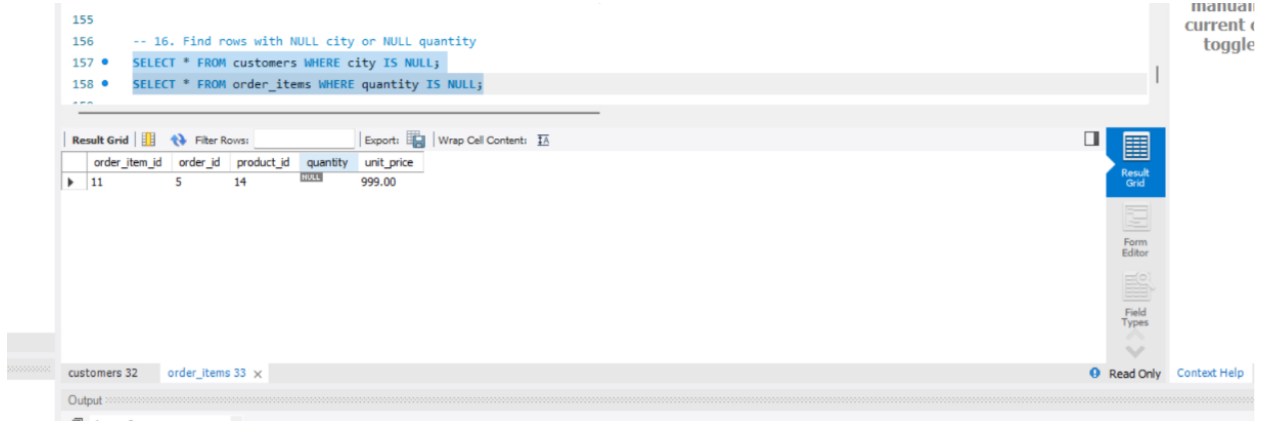
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	orders		ref	idx_orders_customer_id	idx_orders_customer_id	5	const	1	100.00	NULL

Screenshot: Indexes created; EXPLAIN shows MySQL now uses `idx_orders_customer_id` (type = ref) instead of scanning the full orders table

Bonus. Handling NULL Values

Query 16: Find rows with NULL city or NULL quantity

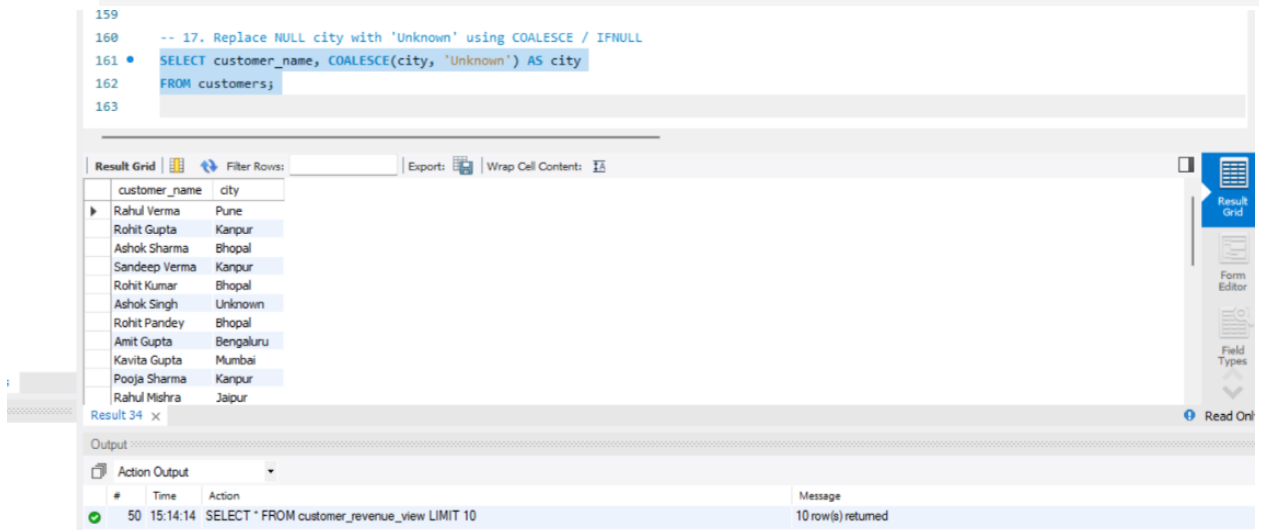
```
SELECT * FROM customers WHERE city IS NULL;  
SELECT * FROM order_items WHERE quantity IS NULL;
```



Screenshot: Output of Query 16 in MySQL Workbench

Query 17: Replace NULL city with 'Unknown' using COALESCE

```
SELECT customer_name, COALESCE(city, 'Unknown') AS city  
FROM customers;
```



Screenshot: Output of Query 17 in MySQL Workbench

Interview Questions & Answers

1. What is the difference between WHERE and HAVING?

WHERE filters individual rows before any grouping happens, and it cannot use aggregate functions like SUM() or COUNT(). HAVING filters groups after GROUP BY has been applied, and it is specifically used to filter on aggregate results, e.g. HAVING SUM(quantity) > 10.

2. What are the different types of joins?

INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table plus matches from the right (unmatched right columns are NULL). RIGHT JOIN does the reverse — all rows from the right table plus matches from the left. FULL OUTER JOIN (not directly supported in MySQL, simulated with UNION of LEFT and RIGHT JOIN) returns all rows from both sides. A CROSS JOIN returns every combination of rows from both tables.

3. How do you calculate average revenue per user in SQL?

First calculate each customer's total revenue by joining orders and order_items and using SUM(quantity * unit_price) grouped by customer_id. Then wrap that result in an outer query and apply AVG() across all the per-customer totals, as shown in Query 11.

4. What are subqueries?

A subquery is a SELECT statement nested inside another query. It can appear in the WHERE clause (to filter based on another result set), in the FROM clause (as a temporary derived table), or as a scalar value in the SELECT list. Subqueries let you break a complex problem into smaller, more readable steps, as shown in Queries 7–9.

5. How do you optimize a SQL query?

Common techniques include: creating indexes on columns used in JOIN, WHERE, and ORDER BY clauses; avoiding SELECT * and only selecting needed columns; using EXPLAIN to see the query execution plan and check whether indexes are actually being used; avoiding unnecessary subqueries when a JOIN would do the same job faster; and filtering data as early as possible in the query.

6. What is a view in SQL?

A view is a saved, virtual table defined by a SELECT query. It doesn't store data itself — it runs the underlying query each time it's accessed. Views are useful for simplifying repeated complex queries (like customer_revenue_view in this task) and for controlling what data users can see.

7. How would you handle null values in SQL?

Use IS NULL / IS NOT NULL to detect or filter them (since NULL can't be compared with =). Use COALESCE() or IFNULL() to substitute a default value when a column is NULL, as shown in Query 17. When doing aggregate calculations like SUM or AVG, remember that most aggregate functions automatically ignore NULLs, but it's still good practice to filter them explicitly when they would distort a calculation (e.g. WHERE quantity IS NOT NULL in this task).